



ENTERPRISE BANKING SYSTEM

Team / Batch: 20 | Section: 10 | Members: Sumanjali – 2520090190, Snigdha – 2520090160, Koumudi - 2520090161

Guide: Ms. Divya Priya | Department: CS&IT | Academic Year: 2026–27

Enterprise web application for authenticated banking workflows, MySQL data management, and transaction fraud analysis.

Abstract

- Web banking prototype combining React, FastAPI, MySQL, and fraud analysis.
- Authenticated access to available account and transaction workflows.
- Academic prototype; real banking rails and live payment processing are outside scope.

Problem Statement & Objectives

- Problem: coordinate secure access, banking data, APIs, and transaction-risk analysis.
- Objective 1: Build a modular banking prototype.
- Objective 2: Implement FastAPI APIs with MySQL.
- Objective 3: Provide authentication and protected access.
- Objective 4: Integrate fraud-detection analysis.
- Objective 5: Test, document, and identify limitations.
- Scope: users, banking workflows, API, database, fraud module.

Existing vs Proposed System

Feature	Existing / Traditional	Proposed System
User access	Manual / fragmented	Web login + protected API
Data	Separate records	Central MySQL database
Transactions	Basic viewing	Structured API workflows
Risk analysis	Limited / manual	Fraud detection component

Feasibility Study

- Technical: React, FastAPI, Python, MySQL, and established libraries.
- Economic: open-source development stack reduces licensing cost.
- Operational: browser-based access with authentication and database configuration.
- Schedule: modular scope supports academic implementation and testing.

Results & Testing

- Backend/API: local service and `./docs` available when running.
- Database: verify environment and schema.
- Authentication: Passlib/bcrypt compatibility issue observed.
- Other tests: verify with synthetic data.

Conclusion

- Combines React, FastAPI, MySQL, and fraud analysis.
- Demonstrates layered full-stack banking design.
- Further testing and dependency fixes remain.

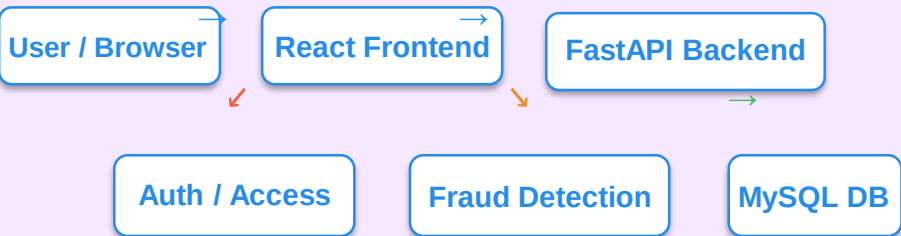
Future Scope

- Complete authentication compatibility and automated testing.
- Role-based access, audit logs, richer transaction history.
- Cloud deployment, monitoring, secure secrets, backups.
- Validated fraud-model evaluation and review workflow.

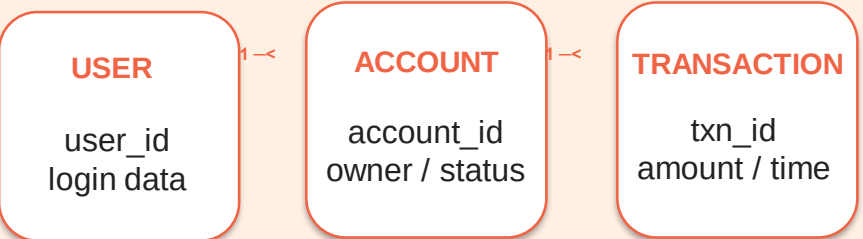
Tools & Technologies

- Front-End: React / HTML-CSS-JS
- Back-End: Python / FastAPI / Uvicorn
- Database: MySQL / SQLAlchemy / PyMySQL
- Authentication: Passlib / python-jose
- Fraud Detection: NumPy / SciPy / scikit-learn
- Testing/API: Browser / Swagger UI / Postman
- Version Control: Git / GitHub (if configured)

System Architecture



ER Diagram & Database Design



Implementation Screenshots

